Introduction: Text Classification with Embeddings + Random Forest

2026-01-12

Introduction

This mini-tutorial shows how to turn unstructured text into a working classifier by combining **semantic embeddings** with a **random forest**. In plain terms: we'll first convert each piece of text into a dense numeric vector that captures its meaning (so "EV battery" ends up near "lithium-ion cell"), then train a tree-based model to predict labels from those vectors (e.g., topic, sentiment, category).

Why this setup? Embeddings give you rich, language-aware features without hand-crafting keywords, while random forests are fast to train, robust to noise, and require little parameter tuning. Together they make a strong, pragmatic baseline for text classification—often "good enough" to ship, and a great foundation to iterate on.

By the end, you'll be able to:

- **Explain** how semantic embeddings represent text in vector space.
- **Build** an end-to-end pipeline: load data → embed text → train a random forest → evaluate.
- **Assess** model quality with accuracy, F1, and a confusion matrix.
- **Deploy** the pipeline to score new, unseen text.

What we'll cover (quickly)

1. Prepare a small labeled text dataset.

2. Generate embeddings with a pre-trained encoder.

3. Train a random-forest classifier on the embeddings.

4. Evaluate and avoid common pitfalls (leakage, class imbalance).

5. Package inference (embed → predict) for real-world use.

Prerequisites

Basic Python or R, a labeled text column with target labels, and common ML libraries (e.g., Python: `sentence-transformers` + `scikit-learn`; or R: `text2vec` + `ranger`). No GPU required, and no deep-learning training—just smart features plus trees.